

TEORÍA DE ALGORITMOS
(TB024) CURSO ECHEVARRÍA

Trabajo Práctico 1



12 de junio de 2026

Integrantes	Padrón
Joaquín Hernández	110470
Juan Francisco Cuevas	107963
Alexander Terrussi	107212
Dante Finci	108456
Víctor Zacarías	107080

Índice

1. Introducción General	4
2. Ejercicio #1: Comparación de Monedas	5
2.1. Problema a resolver	5
2.2. Algoritmo de División y Conquista	5
2.3. Complejidad temporal y cantidad de pesadas	6
2.4. Metodología adoptada	6
2.5. Resultados y gráficos	7
2.6. Relación entre tiempo de ejecución y pesadas	7
3. Ejercicio #2: Algoritmo Paolo	8
3.1. Algoritmo	8
3.2. Estructuras de datos	11
3.3. Naturaleza Greedy del algoritmo	11
3.4. Complejidad temporal y espacial	11
4. Ejercicio #3: Algoritmo Aspiradora Robot	12
4.1. Introducción y Problema	12
4.2. Algoritmo de Backtracking	12
4.3. Supuestos	12
4.4. Diseño	12
4.5. Seguimiento	14
4.6. Resultados experimentales	14
4.7. Complejidad temporal y espacial	15
4.8. Relación con los resultados	15
5. Ejercicio #4: Búsqueda de palíndromos	16
5.1. Supuestos tomados para la resolución del algoritmo	16
5.2. Diseño del Algoritmo	16
5.2.1. Definición del Subproblema	16
5.2.2. Ecuación de Recurrencia	16
5.2.3. Preprocesamiento de Palíndromos	16
5.2.4. Subestructura Óptima	17
5.2.5. Subproblemas Superpuestos	17
5.2.6. Memorización	17
5.2.7. Código en Python	17
5.3. Seguimiento de ejemplo	18
5.4. Complejidad	18
5.5. Sets de Datos	18
5.6. Tiempos de Ejecución	18
5.7. Gráfico de los tiempos de ejecución	18

5.8. Resultados Experimentales	19
5.9. Análisis de los Resultados	19
5.10. Comparación con la Curva Teórica	20
5.11. Conclusión Experimental	20
5.12. Conclusión	20

1. Introducción General

Este informe detalla la resolución de cuatro problemas algorítmicos utilizando diferentes técnicas: División y Conquista, Algoritmos Greedy, Backtracking y Programación Dinámica. Cada sección analiza el problema propuesto, describe el algoritmo diseñado y evalúa su complejidad teórica.

2. Ejercicio #1: Comparación de Monedas

2.1. Problema a resolver

Se tiene una bolsa con n monedas de igual denominación, de las cuales exactamente una es falsa y es más liviana que el resto. El objetivo es identificar la moneda falsa sin ordenar por peso y haciendo uso de División y Conquista.

2.2. Algoritmo de División y Conquista

La entrada del algoritmo es una lista de monedas representadas por su peso. Se asume que todos los pesos son iguales, a excepción del peso de la moneda falsa, es decir: $[b, b, a, b, b]$ donde $a < b$.

En cada paso el algoritmo parte la lista en dos mitades y calcula el **promedio** de cada parte. Usar el promedio en lugar de la suma garantiza correctitud cuando n es impar, ya que ambas mitades pueden tener tamaños distintos.

1. Si el promedio izquierdo es menor al promedio derecho, la moneda falsa está en la mitad izquierda: se recursa sobre ella.
2. Si el promedio derecho es menor al promedio izquierdo, la moneda falsa está en la mitad derecha: se recursa sobre ella.

Casos base:

1. Si la lista tiene longitud uno, se retorna ese único elemento como la moneda falsa.
2. Si la lista tiene longitud dos, se retorna la moneda de menor peso entre los dos elementos.

Pesadas en el algoritmo:

Las pesadas son la cantidad de veces que se usa la balanza para comparar grupos de monedas. En el algoritmo se registra una pesada cada vez que se calculan y comparan los promedios de ambas mitades, y también en los casos base (listas de longitud uno o dos).

Solución:

```
1 Algoritmo EncontrarMonedaFalsaDyC(monedas, inicio, fin, contador)
2
3     // Caso: una sola moneda
4     Si fin = inicio entonces
5         contador := contador + 1
6         Retornar monedas[inicio]
7     FinSi
8
9     // Caso: dos monedas seguidas
10    Si fin - inicio = 1 entonces
11        contador := contador + 1
12
13        Si monedas[inicio] <= monedas[fin] entonces
14            Retornar monedas[inicio]
15        Sino
16            Retornar monedas[fin]
17    FinSi
18    FinSi
19
20    medio := (inicio + fin) / 2
21
22    prom_izq := suma(monedas desde inicio hasta medio - 1) / (medio - inicio)
23    prom_der := suma(monedas desde medio hasta fin - 1) / (fin - medio)
24
25    contador := contador + 1
```

```
26
27     Si prom_izq < prom_der entonces
28         Retornar EncontrarMonedaFalsaDyC(monedas, inicio, medio, contador)
29     Sino
30         Retornar EncontrarMonedaFalsaDyC(monedas, medio, fin, contador)
31     FinSi
32
33 FinAlgoritmo
```

2.3. Complejidad temporal y cantidad de pesadas

El algoritmo es de Divide y Conquista con la siguiente estructura en cada llamada:

- Se divide la lista en dos mitades: $O(1)$.
- Se calcula el promedio de cada mitad con `sum()`: $O(n)$ en total.
- Se recursa sobre **una sola** mitad.

La ecuación de recurrencia resultante es:

$$T(n) = T\left(\frac{n}{2}\right) + O(n)$$

donde $A = 1$ (una sola llamada recursiva), $B = 2$ (el problema se divide a la mitad), y el costo adicional en cada nivel es $O(n)$ por el cálculo de `sum()` sobre ambas mitades.

Aplicando el **Teorema Maestro** con $a = 1$, $b = 2$, $f(n) = n$:

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

Como $f(n) = n = \Omega(n^{0+1})$, se aplica el **Caso 3**: el costo en la raíz del árbol de recursión domina a todos los niveles siguientes. La condición de regularidad se verifica:

$$a \cdot f(n/b) = 1 \cdot \frac{n}{2} \leq \frac{1}{2} \cdot n \quad \checkmark$$

Por lo tanto:

$$T(n) = O(n)$$

Cantidad de pesadas:

Dado que el algoritmo recursa sobre una sola rama en cada nivel, la profundidad del árbol de recursión es $\lceil \log_2 n \rceil$, y se realiza exactamente una pesada por nivel. La cantidad de pesadas sigue:

$$\text{Pesadas}(n) = O(\log_2 n)$$

2.4. Metodología adoptada

Se evaluó el algoritmo con sets de datos de distintos tamaños:

$$n \in \{100, 500, 1000, 5000, 10000, 50000, 100000, 200000, 500000, 1000000\}$$

Para cada tamaño se generó un archivo con n monedas genuinas de peso 10 y una única moneda falsa de peso 4 en una posición aleatoria. El algoritmo se ejecutó con:

```
python3 ejercicios/ej-dyc/monedas.py <tamaño_set_de_datos>
```

Para el análisis del *notebook*, cada tamaño se ejecutó 20 veces y se registró el tiempo promedio, mínimo y máximo con `time.perf_counter()`, junto con la cantidad de pesadas.

2.5. Resultados y gráficos

n	Tiempo (s)	Pesadas
100	0.000057	7
500	0.000096	10
1000	0.000076	11
5000	0.000214	13
10000	0.000358	15
50000	0.001458	17

Cuadro 1: Resultados experimentales del ejercicio 1.

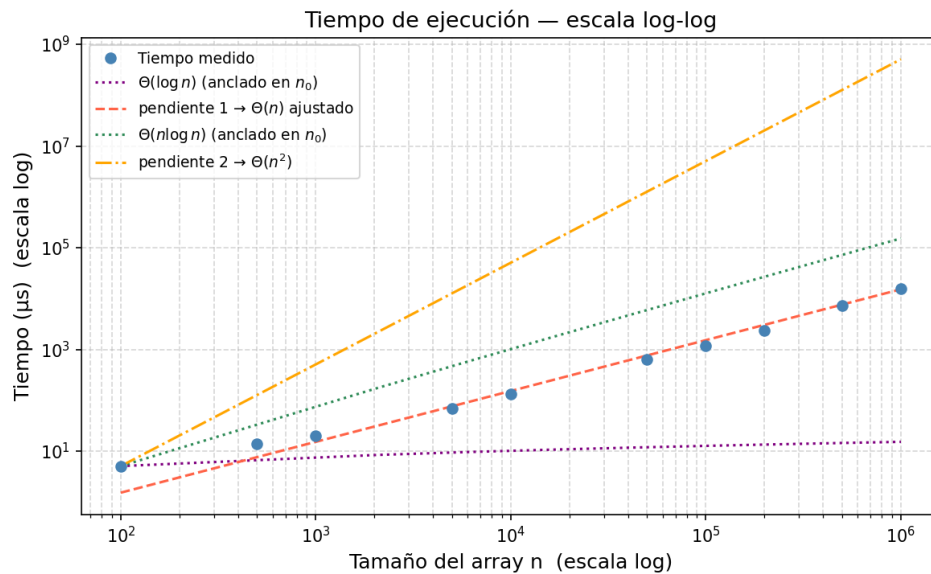


Figura 1: Tiempo de ejecución en escala log-log con curvas de referencia $O(\log n)$, $O(n)$, $O(n \log n)$ y $O(n^2)$. Los datos siguen la recta de pendiente 1, confirmando $T(n) = O(n)$.

Los gráficos confirman el análisis teórico: el tiempo de ejecución crece linealmente con n , mientras que la cantidad de pesadas crece logarítmicamente.

2.6. Relación entre tiempo de ejecución y pesadas

El tiempo de ejecución y la cantidad de pesadas tienen **complejidades distintas**:

- **Tiempo:** $O(n)$, dominado por el cómputo de `sum()` en cada nivel de recursión. Aunque solo se recursa una rama, el cálculo de los promedios recorre $n/2$ elementos en el primer nivel, $n/4$ en el segundo, etc., sumando en total $\approx 2n$ operaciones.
- **Pesadas:** $O(\log_2 n)$, ya que se realiza exactamente una comparación de grupos por nivel, y el árbol tiene profundidad $\lceil \log_2 n \rceil$.

La distinción es relevante según qué se considere la *operación elemental*: si se modela el problema con una balanza de platillos donde pesar un grupo completo cuenta como una sola operación, el costo relevante es $O(\log n)$ pesadas. Si se considera el costo computacional real (calcular la suma), el costo es $O(n)$.

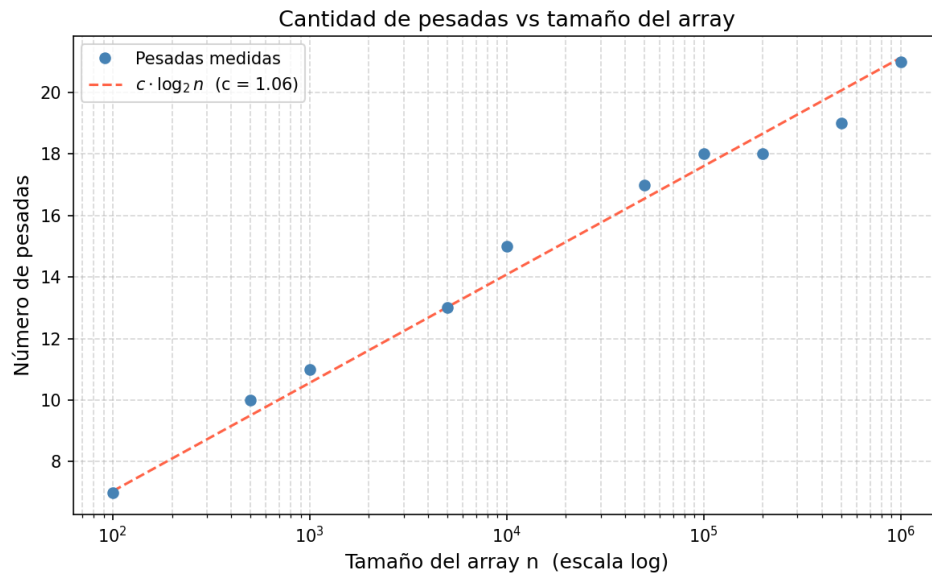


Figura 2: Cantidad de pesadas vs tamaño del array en escala semi-log. Los puntos se alinean sobre la curva $c \cdot \log_2 n$, confirmando $\text{Pesadas}(n) = O(\log_2 n)$.

3. Ejercicio #2: Algoritmo Paolo

Se tiene un algoritmo escrito en un lenguaje inusual, el objetivo es el de identificar el problema que resuelve, analizando su comportamiento greedy, el uso de estructuras de datos y la complejidad del mismo.

Adicionalmente, el algoritmo será traducido a un lenguaje de programación más moderno, Python.

3.1. Algoritmo

El problema resuelto por el algoritmo es el de caminos mínimos desde un único origen en un grafo no dirigido con pesos no negativos. El mismo fue implementado con el siguiente código:

```
1 10 CLS
2 20 INPUT "INGRESE EL NUMERO DE VERTICES "; N
3 30 PRINT
4 40 DIM COST (N,N)
5 50 DIM DIST (N)
6 60 DIM SOL (N)
7 70 A=1
8 80 PRINT "INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)"
9 90 PRINT
10 100 INPUT "EL EJE "; A, B
11 110 IF A=0 THEN 140
12 120 INPUT "COSTO DEL EJE "; COST(A,B)
13 130 GOTO 100
14 140 FOR I=1 TO N
15 150 FOR J=1 TO N
16 160 IF COST (I,J)=0 THEN COST (I,J)=15000
17 170 NEXT J
18 180 NEXT I
19 190 FOR I=1 TO N
20 200 FOR J=1 TO I
21 210 COST (I,J)=COST (J,I)
22 220 NEXT J
23 230 NEXT I
24 240 INPUT "INGRESE EL VERTICE DE SALIDA "; V
```



```
25 250 FOR I=1 TO N
26 260 DIST(I)=COST(V,I)
27 270 SOL(I)=0
28 280 NEXT I
29 290 GOSUB 1000
30 300 PRINT "SALIDA","LLEGADA","DISTANCIA"
31 310 FOR I=1 TO N
32 320 IF DIST(I)<15000 THEN PRINT V, I, DIST(I)
33 330 NEXT I
34 340 INPUT "OTRA VEZ? (SI/NO) "; RES$
35 350 IF RES$="NO" THEN END
36 360 FOR I=1 TO N
37 370 SOL(I)=0: DIST(I)=0
38 380 NEXT I
39 390 GOTO 240
40 1000 SOL(V)=1
41 1010 DIST(V)=0
42 1020 FOR I=1 TO N-1
43 1030 U=15000
44 1040 FOR J=1 TO N
45 1050 IF DIST(J)<=U AND SOL(J)=0 THEN U=J
46 1060 NEXT J
47 1070 SOL(U)=1
48 1080 FOR J=1 TO N
49 1090 IF DIST(J)>= (DIST(U)+COST(U,J)) THEN DIST(J)=DIST(U)+COST(U,J)
50 1100 NEXT J
51 1110 NEXT I
52 1120 RETURN
```

Listing 1: Algoritmo Paolo original

Su traducción más literal a un lenguaje moderno (Python) es:

```
1 Algoritmo Paolo
2
3     Leer n
4
5     Crear matriz cost de n x n inicializada en 0
6     Crear vector dist de tamaño n inicializado en 0
7     Crear vector sol de tamaño n inicializado en 0
8
9     Mostrar "Ingrese el cuadro de costos. Ingrese 0,0 para terminar"
10
11     Mientras Verdadero hacer
12         Leer a, b
13
14         Si a = 0 y b = 0 entonces
15             Cortar
16         FinSi
17
18         Leer costo_eje
19         cost[a][b] := costo_eje
20     FinMientras
21
22     Para i := 0 hasta n - 1 hacer
23         Para j := 0 hasta n - 1 hacer
24             Si cost[i][j] = 0 entonces
25                 cost[i][j] := 15000
26             FinSi
27         FinPara
28     FinPara
29
30     Para i := 0 hasta n - 1 hacer
31         Para j := 0 hasta i - 1 hacer
32             cost[i][j] := cost[j][i]
33         FinPara
34     FinPara
35
36     Leer v
37
38     Para i := 0 hasta n - 1 hacer
```

```
39     dist[i] := cost[v][i]
40     sol[i] := 0
41   FinPara
42
43   PaoloSub(n, cost, dist, sol, v)
44
45   Mostrar "SALIDA", "LLEGADA", "DISTANCIA"
46
47   Para i := 0 hasta n - 1 hacer
48     Si dist[i] < 15000 entonces
49       Mostrar v, i, dist[i]
50     FinSi
51   FinPara
52
53   Leer res
54
55   Si res = "NO" entonces
56     Terminar
57   FinSi
58
59   Para i := 0 hasta n - 1 hacer
60     sol[i] := 0
61     dist[i] := 0
62   FinPara
63
64   Paolo
65
66 FinAlgoritmo
67
68 Algoritmo PaoloSub(n, cost, dist, sol, v)
69
70   sol[v] := 1
71   dist[v] := 0
72
73   Para i := 0 hasta n - 2 hacer
74
75     u := 15000
76
77     Para j := 0 hasta n - 1 hacer
78       Si dist[j] <= u y sol[j] = 0 entonces
79         u := j
80       FinSi
81     FinPara
82
83     sol[u] := 1
84
85     Para j := 0 hasta n - 1 hacer
86       Si dist[j] >= dist[u] + cost[u][j] entonces
87         dist[j] := dist[u] + cost[u][j]
88       FinSi
89     FinPara
90
91   FinPara
92
93 FinAlgoritmo
```

El algoritmo crea una matriz de adyacencia para representar al grafo, donde cada entrada $cost[i][j]$ almacena el costo de viajar desde el vértice i al vértice j . Si un costo no se ingresa, se asume un valor predeterminado de 15000 (considerado infinito en este contexto). Luego, la matriz se ajusta para que sea simétrica, lo que indica que el grafo es no dirigido.

A partir de un vértice de salida v , calcula las distancias más cortas a todos los demás vértices utilizando una variante del algoritmo de Dijkstra. En cada iteración, selecciona el vértice no visitado con menor distancia y actualiza las distancias del resto usando ese vértice como punto de partida.

3.2. Estructuras de datos

Se utilizan una matriz de adyacencia y dos arreglos. La matriz *cost* permite consultar en tiempo constante el peso de cualquier arista, el arreglo *dist* guarda las mejores distancias encontradas hasta el momento, y el arreglo *sol* indica qué vértices ya fueron incorporados definitivamente a la solución.

3.3. Naturaleza Greedy del algoritmo

En cada paso del algoritmo se toma la misma decisión, se elige el vértice no visitado con menor distancia desde el origen, esta regla constituye una regla Greedy ya que se está tomando una decisión en base a un óptimo local (¿cuál es la menor distancia a la que me puedo mover”), la cual en conjunto al resto de pasos del algoritmo que replican la misma regla buscan acercarse al óptimo global.

3.4. Complejidad temporal y espacial

La complejidad temporal es $O(n^2)$ y la complejidad espacial es $O(n^2)$. El uso de memoria está dominado por la matriz de adyacencia de tamaño $n \times n$. En tiempo, el algoritmo realiza para cada uno de los n vértices una búsqueda lineal del siguiente vértice a fijar y una recorrida lineal para mejorar las distancias, resultando en un costo cuadrático.

4. Ejercicio #3: Algoritmo Aspiradora Robot

4.1. Introducción y Problema

En este ejercicio se modela el recorrido de una aspiradora robot dentro de un laberinto rectangular. El laberinto se representa mediante una matriz, donde cada celda puede corresponder a una pared (X), una posición transitable, la entrada (E) o la salida (S).

El objetivo es diseñar, implementar y documentar un algoritmo de backtracking que, dado un laberinto de tamaño $m \times n$, encuentre un camino válido desde la posición inicial E hasta la salida S , en caso de que dicho camino exista.

La solución devuelve el recorrido como una secuencia de coordenadas de la matriz. A partir de ese camino, pueden interpretarse los movimientos que debería realizar la aspiradora para desplazarse por el laberinto, utilizando sus operaciones disponibles: observar la celda siguiente, avanzar y girar.

4.2. Algoritmo de Backtracking

El problema se resuelve mediante backtracking. Desde la entrada E , el algoritmo intenta avanzar recursivamente hacia las posiciones vecinas en un orden fijo: derecha, izquierda, abajo y arriba.

Antes de avanzar, se verifica que la celda esté dentro de los límites, que no sea una pared X y que no haya sido visitada previamente. Si la celda es válida, se agrega a la solución y se marca como visitada.

Cuando se alcanza la salida S , se devuelve el camino encontrado. Si desde una posición no se puede continuar hacia la salida, se elimina esa posición de la solución y se vuelve atrás para probar otra alternativa.

4.3. Supuestos

Se asume que el laberinto es rectangular, por lo que todas sus filas tienen la misma cantidad de columnas. Sus dimensiones se obtienen a partir del archivo de entrada.

También se asume que existe una única posición inicial E y al menos una salida S . Las celdas marcadas con X representan paredes y no pueden ser recorridas. Del mismo modo, cualquier posición fuera de los límites del laberinto se considera inválida.

En caso de que no exista un camino posible entre la entrada y la salida, el algoritmo devuelve una lista vacía.

4.4. Diseño

La solución se basa en un algoritmo recursivo de backtracking. En cada llamada se analiza una posición del laberinto y se verifica si puede ser recorrida. Una posición es válida si está dentro de los límites de la matriz, no corresponde a una pared y no fue visitada anteriormente.

Si la posición es válida, se agrega a la solución y se marca como visitada. Luego se prueban las cuatro direcciones posibles en un orden fijo: derecha, izquierda, abajo y arriba. Si alguna de esas ramas alcanza la salida, el algoritmo termina y devuelve el camino encontrado. En caso contrario, se elimina la posición actual de la solución y se retrocede para probar otras alternativas.

Las estructuras de datos utilizadas son:

- Una matriz de caracteres para representar el laberinto.
- Una lista con la solución construida hasta el momento.
- Un conjunto de posiciones visitadas para evitar ciclos y no recorrer dos veces la misma celda.

Código en Python:

```
1 DIRECCIONES = (  
2     (0, 1),  
3     (0, -1),  
4     (1, 0),  
5     (-1, 0),  
6 )  
7  
8  
9 def obtener_inicio(laberinto):  
10     if not laberinto:  
11         return None, None  
12  
13     for fila in range(len(laberinto)):  
14         for columna in range(len(laberinto[0])):  
15             if laberinto[fila][columna] == "E":  
16                 return fila, columna  
17  
18     return None, None  
19  
20  
21 def posicion_valida(laberinto, fila, columna, visitados):  
22     return (  
23         fila >= 0  
24         and columna >= 0  
25         and fila < len(laberinto)  
26         and columna < len(laberinto[0])  
27         and laberinto[fila][columna] != "X"  
28         and (fila, columna) not in visitados  
29     )  
30  
31  
32 def avanzar(laberinto, fila, columna, resultado, visitados):  
33     if not posicion_valida(laberinto, fila, columna, visitados):  
34         return False  
35  
36     resultado.append((fila, columna))  
37     visitados.add((fila, columna))  
38  
39     if laberinto[fila][columna] == "S":  
40         return True  
41  
42     for desplazamiento_fila, desplazamiento_columna in DIRECCIONES:  
43         nueva_fila = fila + desplazamiento_fila  
44         nueva_columna = columna + desplazamiento_columna  
45  
46         if avanzar(laberinto, nueva_fila, nueva_columna, resultado, visitados):  
47             return True  
48  
49     resultado.pop()  
50     return False  
51  
52  
53 def encontrar_camino_bt(laberinto):  
54     fila_inicial, columna_inicial = obtener_inicio(laberinto)  
55  
56     if fila_inicial is None or columna_inicial is None:  
57         return []  
58  
59     resultado = []  
60     visitados = set()  
61  
62     encontro_salida = avanzar(  
63         laberinto,  
64         fila_inicial,  
65         columna_inicial,  
66         resultado,  
67         visitados,  
68     )  
69
```

```
70     if encontro_salida:
71         return resultado
72
73     return []
```

El código del algoritmo se encuentra en *code/laberinto.py*. La función *avanzar* concentra la lógica recursiva del backtracking, mientras que *encontrar_camino_bt* busca la posición inicial y actúa como punto de entrada del algoritmo.

4.5. Seguimiento

Para el seguimiento se toma el archivo *laberinto1.txt*.

Las posiciones se indican con índices base cero. Por lo tanto, la entrada *E* se encuentra en la posición (5, 1) y la salida *S* en la posición (1, 7).

En el laberinto analizado, la entrada *E* se encuentra en la posición (5, 1) y la salida *S* en la posición (1, 7).

A partir de ahí, continúa explorando recursivamente las posiciones válidas. Cada celda recorrida se agrega a la solución y se marca como visitada. Cuando una rama no permite llegar a la salida, el algoritmo retrocede y prueba otra dirección.

El camino devuelto para este caso es:

```
[(5, 1), (4, 1), (3, 1), (3, 2), (3, 3), (4, 3), (4, 4),
(4, 5), (4, 6), (3, 6), (2, 6), (1, 6), (1, 7)]
```

Este resultado muestra que el algoritmo encuentra un camino válido desde la entrada hasta la salida, sin atravesar paredes ni repetir celdas.

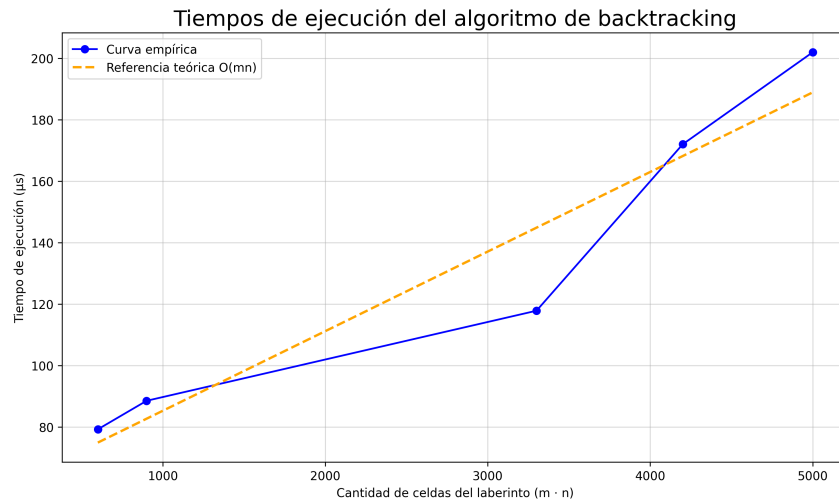
4.6. Resultados experimentales

Para validar el comportamiento del algoritmo, se ejecutó la solución sobre cinco conjuntos de datos con laberintos de distinto tamaño. En todos los casos, el algoritmo finalizó correctamente y devolvió un camino válido desde la entrada *E* hasta la salida *S*. Para las mediciones se utilizó el módulo *ejercicios/ej-backtracking/laberinto_informe.py*, que imprime el tiempo de ejecución.

Los tiempos de ejecución obtenidos fueron los siguientes:

Caso	Tiempo de ejecución
Laberinto 1	79,209 μs
Laberinto 2	88,500 μs
Laberinto 3	117,834 μs
Laberinto 4	172,042 μs
Laberinto 5	202,000 μs

A continuación se presenta un gráfico comparativo de los tiempos medidos para cada conjunto de datos.



Como se observa en la tabla y en el gráfico, los tiempos de ejecución presentan una tendencia creciente a medida que aumenta el tamaño de los laberintos. Esto resulta coherente con el análisis teórico, ya que para entradas más grandes el algoritmo puede requerir explorar una mayor cantidad de posiciones antes de encontrar la salida.

4.7. Complejidad temporal y espacial

Sea m la cantidad de filas del laberinto y n la cantidad de columnas. En el peor caso, el algoritmo puede llegar a analizar todas las celdas de la matriz.

Como las posiciones visitadas se almacenan en un conjunto, cada celda se analiza a lo sumo una vez y la consulta para saber si ya fue visitada tiene costo promedio $O(1)$. Para cada posición válida se prueban hasta cuatro direcciones posibles: derecha, izquierda, abajo y arriba. Como la cantidad de direcciones es constante, la complejidad temporal resulta:

$$O(mn)$$

La complejidad espacial también es $O(mn)$. Esto se debe a que el algoritmo almacena la solución y el conjunto de posiciones visitadas. En el peor caso, ambas estructuras pueden crecer proporcionalmente a la cantidad total de celdas del laberinto.

Por lo tanto, tanto la complejidad temporal como la espacial del algoritmo son $O(mn)$.

4.8. Relación con los resultados

Los tiempos medidos son bajos, por lo que pueden verse afectados en parte por factores propios de la ejecución y de la medición. Sin embargo, al considerar los cinco conjuntos de datos, se observa una tendencia creciente a medida que aumenta el tamaño del laberinto.

Estos resultados son compatibles con la complejidad temporal, $O(mn)$, ya que el algoritmo recorre como máximo una vez cada celda del laberinto y, para cada una, prueba una cantidad constante de direcciones.

De todos modos, el tiempo de ejecución no depende únicamente de la cantidad total de celdas, sino también de la distribución de paredes y de la ubicación de la salida. Si la salida se encuentra rápidamente, el algoritmo puede finalizar sin recorrer toda la matriz. En cambio, si el camino requiere explorar más alternativas, el tiempo aumenta.

Por lo tanto, los resultados experimentales acompañan el análisis teórico: en los casos evaluados, al aumentar el tamaño del laberinto también aumenta el tiempo de ejecución, manteniéndose dentro del comportamiento esperado para una búsqueda con backtracking sobre una matriz.

5. Ejercicio #4: Búsqueda de palíndromos

Para este ejercicio se busca resolver mediante programación dinámica el problema de obtener la menor cantidad de palíndromos dentro de un string.

5.1. Supuestos tomados para la resolución del algoritmo

- El algoritmo tiene como entrada un string longitud $n \geq 1$.
- Un string de longitud 1 es un palíndromo, por ejemplo, *A* es un palíndromo.
- El acceso a caracteres es $O(1)$ dentro del string.

5.2. Diseño del Algoritmo

Para la resolución del algoritmo mediante programación dinámica fue necesario analizar en primer lugar la forma que tienen los subproblemas del problema a resolver, y en segundo la ecuación de recurrencia del mismo.

5.2.1. Definición del Subproblema

Sea:

$dp[i]$ = mínima cantidad de palíndromos necesarios para $S[0..i]$,

$S[0..i]$: El string desde la posición 0 hasta la posición i .

5.2.2. Ecuación de Recurrencia

$$dp[i] = \min_{0 \leq j \leq i} \{dp[j-1] + 1 \mid \text{Si } S[j..i] \text{ es palíndromo}\}$$

Condición base:

$$dp[-1] = 0$$

Nota: Para una string que va desde 0 hasta i la ecuación de recurrencia busca obtener la mínima cantidad de palíndromos que hay en todos los substrings que hay desde 0 hasta i .

5.2.3. Preprocesamiento de Palíndromos

Como paso inicial, se define una matriz:

$$pal[i][j] = \begin{cases} True & \text{si } S[i..j] \text{ es palíndromo} \\ False & \text{en caso contrario} \end{cases}$$

$$pal[i][j] = (S[i] = S[j]) \wedge (j - i \leq 1 \vee pal[i+1][j-1])$$

Esta permite evaluar si el string que va desde i hasta j es un palíndromo.

La matriz sirve como estructura de datos auxiliar para el cálculo de la cantidad mínima de palíndromos para cada tamaño de subproblemas, evitando realizar cálculos de forma repetitiva.

5.2.4. Subestructura Óptima

La solución óptima para $S[0..i]$ depende de soluciones óptimas de subproblemas $S[0..j - 1]$. Esto cumple con la propiedad de la programación dinámica de resolver un problema a partir del espacio de subproblemas del mismo.

5.2.5. Subproblemas Superpuestos

Los valores de $dp[i]$ y $pal[i][j]$ se reutilizan múltiples veces, evitando recomputaciones.

5.2.6. Memorización

Se utilizan:

- Matriz $pal[n][n]$
- Lista $dp[n]$

La matriz, tal como se comentó en la sección "Preprocesamiento de Palíndromos" se usa para almacenar que substrings del string original son palíndromos. La lista de óptimos almacena la mínima cantidad de palíndromos para todos los substrings de la forma $[0...i]$.

La principal diferencia con la matriz, es que la matriz guarda todas las combinaciones de substring, y la ecuación de recurrencia solo los que van de 0 hasta i (subproblemas del algoritmo).

5.2.7. Código en Python

```
1 Algoritmo MinPalindromos(S)
2
3     n := longitud(S)
4
5     Si n = 0 entonces
6         Retornar 0
7     FinSi
8
9     Crear matriz pal de n x n inicializada en Falso
10
11     Para i := 0 hasta n - 1 hacer
12         pal[i][i] := Verdadero
13     FinPara
14
15     Para largo := 2 hasta n hacer
16         Para i := 0 hasta n - largo hacer
17             j := i + largo - 1
18
19             Si S[i] = S[j] y (largo = 2 o pal[i + 1][j - 1]) entonces
20                 pal[i][j] := Verdadero
21             FinSi
22         FinPara
23     FinPara
24
25     Crear vector dp de tamaño n inicializado en 0
26
27     Para i := 0 hasta n - 1 hacer
28
29         Si pal[0][i] entonces
30             dp[i] := 1
31         Sino
32             dp[i] := infinito
33
34         Para j := 1 hasta i hacer
35             Si pal[j][i] entonces
36                 dp[i] := minimo(dp[i], dp[j - 1] + 1)
```

```
37         FinSi
38         FinPara
39     FinSi
40
41     FinPara
42
43     Retornar dp[n - 1]
44
45 FinAlgoritmo
```

5.3. Seguimiento de ejemplo

Ejemplo: $S = ABA$

- $dp[0] = 1$
- $dp[1] = 2$
- $dp[2] = 1$

Resultado: 1 partición.

5.4. Complejidad

Para analizar la complejidad del algoritmo es necesario considerar la complejidad incurrida en cada parte del mismo. Se cuentan con dos partes principales, el armado de la matriz y el cálculo de los óptimos a partir de la ecuación de recurrencia.

- Cálculo de palíndromos: $O(n^2)$
- Programación dinámica: $O(n^2)$

Complejidad total:

$$O(n^2)$$

5.5. Sets de Datos

Se utilizaron los siguientes tipos de datos para estudiar al algoritmo:

- Mejor caso: cadenas repetidas (ej: AAAA)
- Peor caso: caracteres distintos (ej: ABCDEF)
- Casos aleatorios con semilla fija

5.6. Tiempos de Ejecución

Se midieron tiempos utilizando múltiples ejecuciones por cada tamaño de entrada, promediando los resultados para cada tamaño de entrada.

5.7. Gráfico de los tiempos de ejecución

Se obtuvo un crecimiento cuadrático en los tiempos de ejecución, consistente con la complejidad teórica.

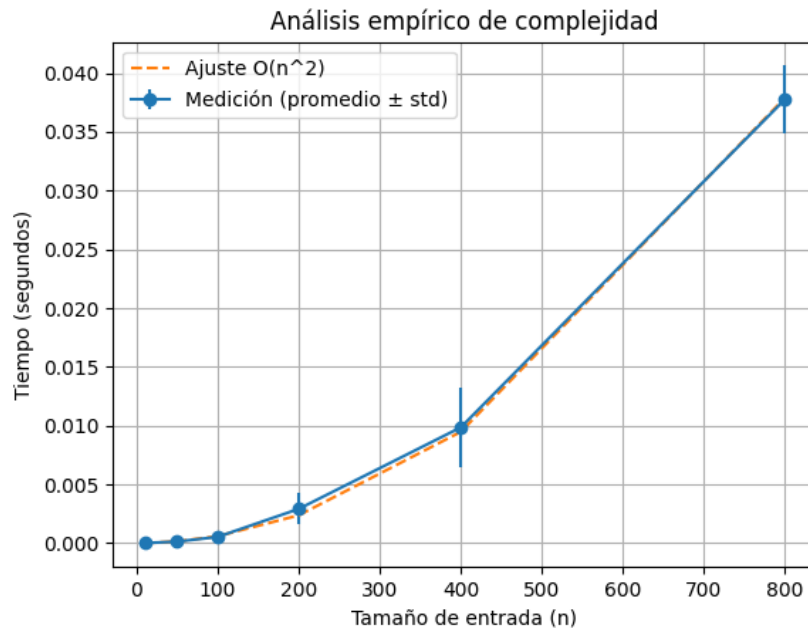


Figura 3: Comparación entre tiempos medidos y curva teórica

5.8. Resultados Experimentales

A continuación se presentan los resultados obtenidos a partir de la ejecución del algoritmo sobre distintos tamaños de entrada:

n	Resultado	Tiempo Promedio (s)
10	5	0.00000941
50	13	0.00013710
100	26	0.00053040
200	61	0.00291414
400	127	0.00985342
800	261	0.03777179

5.9. Análisis de los Resultados

A partir de los datos obtenidos, se observa que el tiempo de ejecución crece de manera consistente al aumentar el tamaño de la entrada. En particular, el crecimiento no es lineal, sino que se acelera a medida que n aumenta, lo cual es indicativo de un comportamiento cuadrático.

Este resultado es coherente con el análisis teórico del algoritmo, cuya complejidad temporal es $O(n^2)$. Si se comparan los tiempos entre distintas escalas, se puede notar que al duplicar el tamaño de la entrada, el tiempo de ejecución tiende a multiplicarse aproximadamente por un factor cercano a 4, lo cual coincide con el comportamiento esperado de una función cuadrática.

Por ejemplo, al pasar de $n = 200$ a $n = 400$, el tiempo crece de aproximadamente 0,0029 segundos a 0,0098 segundos, lo cual representa un incremento cercano a cuatro veces. De manera similar, al duplicar nuevamente hasta $n = 800$, el tiempo alcanza aproximadamente 0,0377 segundos, manteniendo la misma tendencia.

5.10. Comparación con la Curva Teórica

Al comparar los tiempos medidos con la curva teórica de complejidad $O(n^2)$, se observa que ambos presentan una tendencia similar. Esto valida empíricamente el análisis teórico realizado previamente.

Si bien pueden existir pequeñas desviaciones, especialmente en tamaños pequeños donde los costos constantes tienen mayor impacto, en general la curva experimental se ajusta adecuadamente al modelo teórico.

5.11. Conclusión Experimental

Los resultados obtenidos confirman que el algoritmo presenta un comportamiento cuadrático en la práctica, tal como se había deducido teóricamente. Esto demuestra la efectividad del enfoque de programación dinámica utilizado, tanto desde el punto de vista conceptual como en términos de rendimiento.

5.12. Conclusión

A lo largo de este trabajo se pudo ver cómo un problema que en principio parece complejo, como probar todas las formas posibles de dividir una cadena, puede resolverse de manera eficiente utilizando programación dinámica.

La clave estuvo en cambiar la forma de pensar el problema: en lugar de evaluar todas las particiones posibles, se aprovechó la estructura del problema para construir la solución a partir de subproblemas más pequeños. Además, el preprocesamiento de palíndromos permitió evitar cálculos repetidos y mejorar significativamente el rendimiento.

En la práctica, los resultados obtenidos muestran que el tiempo de ejecución crece de manera cuadrática con el tamaño de la entrada, lo cual coincide con el análisis teórico realizado. Si bien para tamaños pequeños la diferencia no es tan evidente, a medida que crece el tamaño de la cadena se observa claramente esta tendencia.

En conclusión, el uso de programación dinámica permitió transformar un problema potencialmente exponencial en uno manejable, demostrando la importancia de identificar subestructuras óptimas y reutilizar resultados ya calculados.